

Algorithms

Lecture #39

Syed Hamed Raza

Dijkstra's Algorithm

Dijkstra's Algorithm

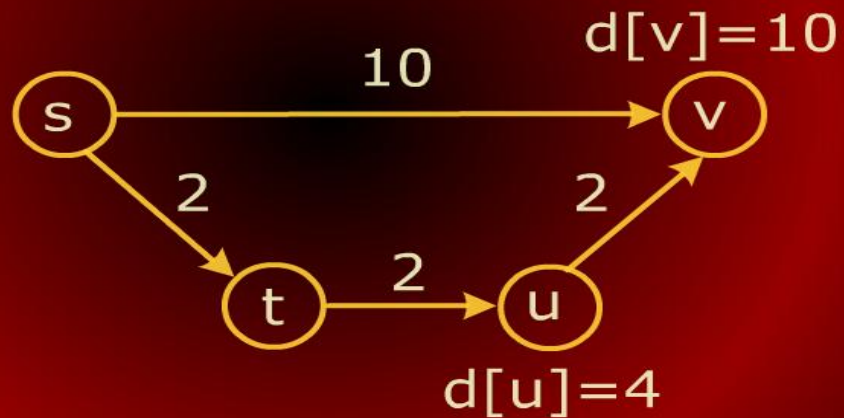
Dijkstra's Algorithm

- As the algorithm goes on and sees more and more vertices, it attempts to update $d[v]$ for each vertex in the graph.
- The process of updating estimates is called relaxation.
- The algorithm stops when all $d[v]$ values converge to the true shortest distances.

Dijkstra's Algorithm

Dijkstra's Algorithm

Relaxation: $d[u]$ updated

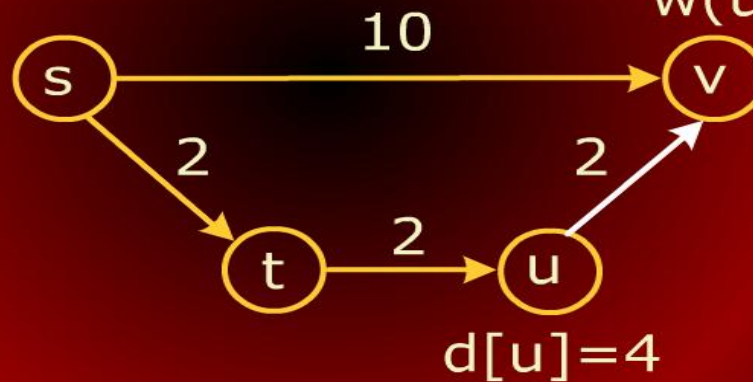


Dijkstra's Algorithm

Dijkstra's Algorithm

Relaxation: $d[u]$ updated

$$d[v] = d[u] + w(u, v) = 6$$



Dijkstra's Algorithm

Dijkstra's Algorithm

RELAX((**u**, **v**))

- 1** if ($d[u] + w(u, v) < d[v]$)
- 2** then $d[v] \leftarrow d[u] + w(u, v)$
- 3** $\text{pred}[v] = u$

Dijkstra's Algorithm

Dijkstra's Algorithm

- Dijkstra's algorithm is based on the notion of performing repeated relaxations.
- Dijkstra's algorithm operates by maintaining a subset of vertices, $S \subseteq V$, for which we claim we know the true distance,
- $d[v] = \delta(s, v)$.

Dijkstra's Algorithm

Dijkstra's Algorithm

- Initially $S = \phi$, the empty set.
- We set $d[u] = 0$ and all others to ∞ .
- One by one we select vertices from $V-S$ to add to S .

Dijkstra's Algorithm

Dijkstra's Algorithm

- How do we select which vertex among the vertices of $V-S$ to add next to S ?
- Here is greediness comes in.
- For each vertex $u \in (V-S)$, we have computed a distance estimate $d[u]$.

Dijkstra's Algorithm

Dijkstra's Algorithm

- The greedy thing to do is to take the vertex for which $d[u]$ is minimum,
- i.e., take the unprocessed vertex that is closest by our estimate to s .
- Later, we justify why this is the proper choice.
- In order to perform this selection efficiently, we store the vertices of $V-S$ in a priority queue.

Dijkstra's Algorithm

Dijkstra's Algorithm

DIJKSTRA((G,w, s))

```
1  for ( each  $u \in V$  )
2  do  $d[u] \leftarrow \infty$ 
3      pq.insert (u,  $d[u]$ )
4   $d[s] \leftarrow 0$ ;  $pred[s] \leftarrow nil$ ;
   pq.decrease_key(s,  $d[s]$ );
5  while ( pq.not_empty () )
6  do  $u \leftarrow pq.extract\_min ()$ 
7      for ( each  $v \in adj[u]$  )
8      do if ( $d[u] + w(u, v) < d[v]$ )
9          then  $d[v] = d[u] + w(u, v)$ 
10             pq.decrease_key (v,  $d[v]$ )
11             pred [v] = u
```

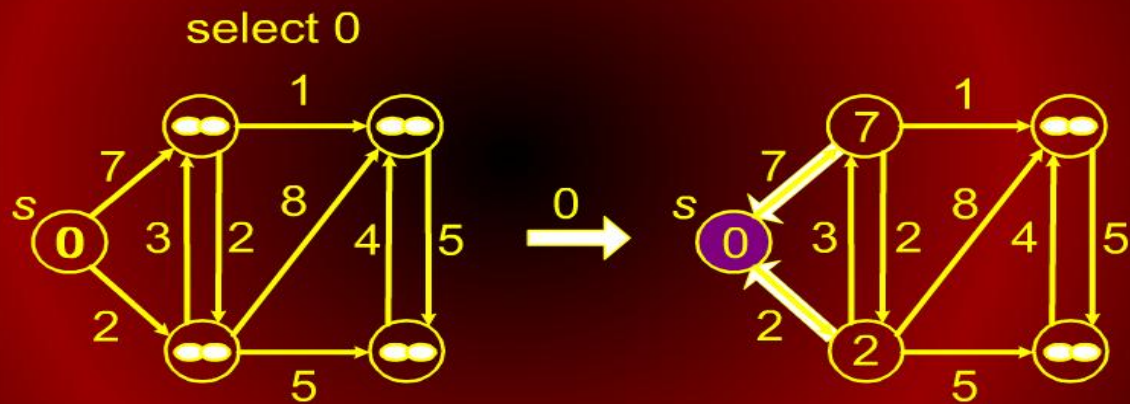
Dijkstra's Algorithm

Dijkstra's Algorithm

- Note the similarity with Prim's algorithm, although a different key is used here.
- Therefore the running time is the same, i.e., $\Theta(E \log V)$.

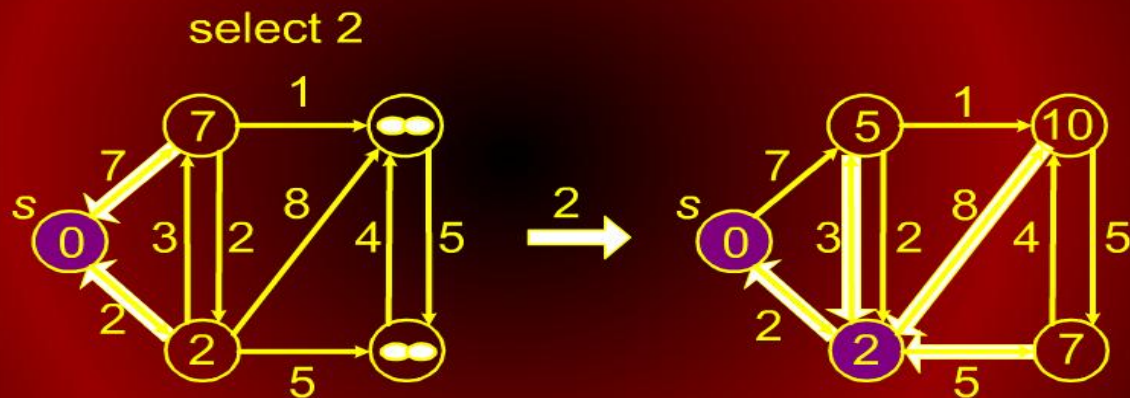
Dijkstra's Algorithm

Dijkstra's Algorithm



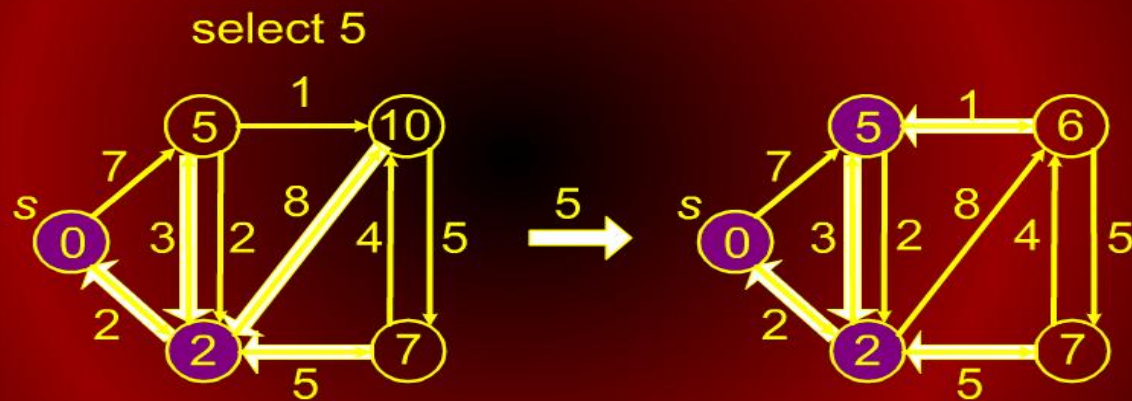
Dijkstra's Algorithm

Dijkstra's Algorithm



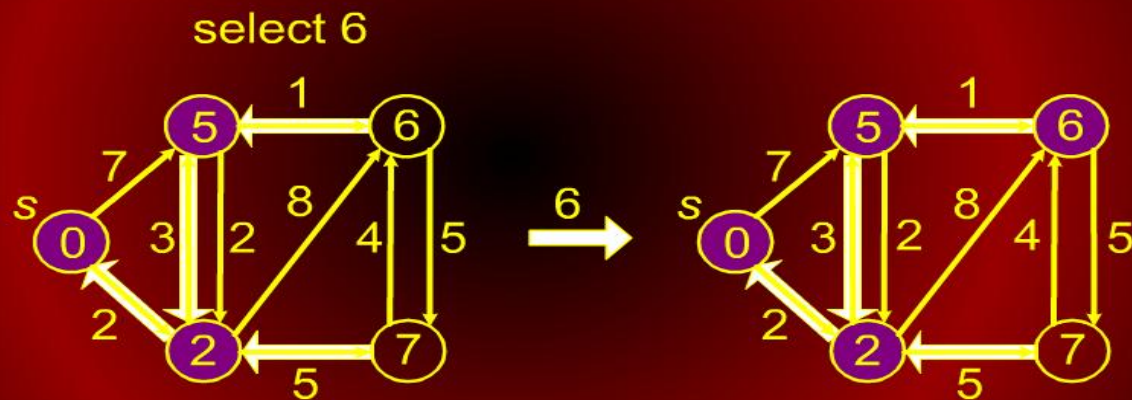
Dijkstra's Algorithm

Dijkstra's Algorithm



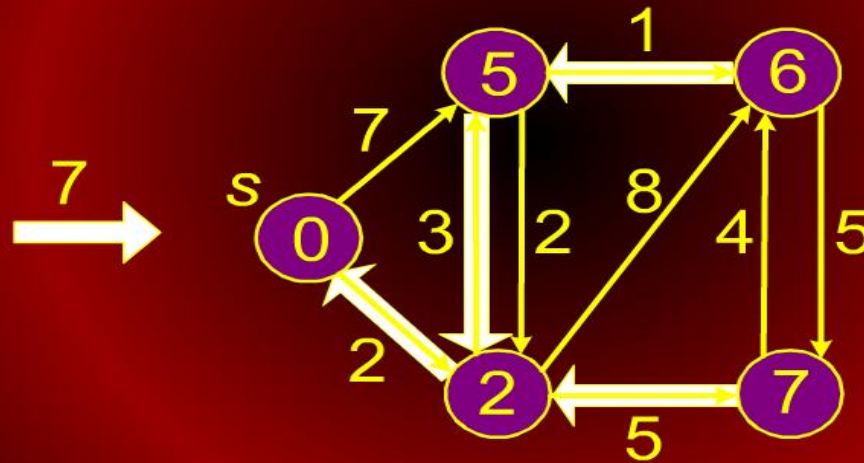
Dijkstra's Algorithm

Dijkstra's Algorithm



Dijkstra's Algorithm

Dijkstra's Algorithm
select 7



Dijkstra's Algorithm – Correctness

Dijkstra's Algorithm - Correctness

Proof of Minimality by Induction

Definitions

$\delta(s, v)$: the minimal distance
from s to v .

Base case

1. $S = \{s\}$
2. $d(s) = 0$, which is $\delta(s, s)$

Dijkstra's Algorithm – Correctness

Dijkstra's Algorithm - Correctness

Assume

1. $d(v) = \delta(s, v)$ for every $v \in S$
2. All neighbors of v have been relaxed.

Dijkstra's Algorithm – Correctness

Dijkstra's Algorithm - Correctness

Proof

- If $d(u) \leq d(u')$ for every $u' \in V$ then $d(u) = \delta(s, u)$, and we can transfer u from V to S , after which $d(v) = \delta(s, v)$ for every $v \in S$.
- We do this as a proof by contradiction.

Dijkstra's Algorithm – Correctness

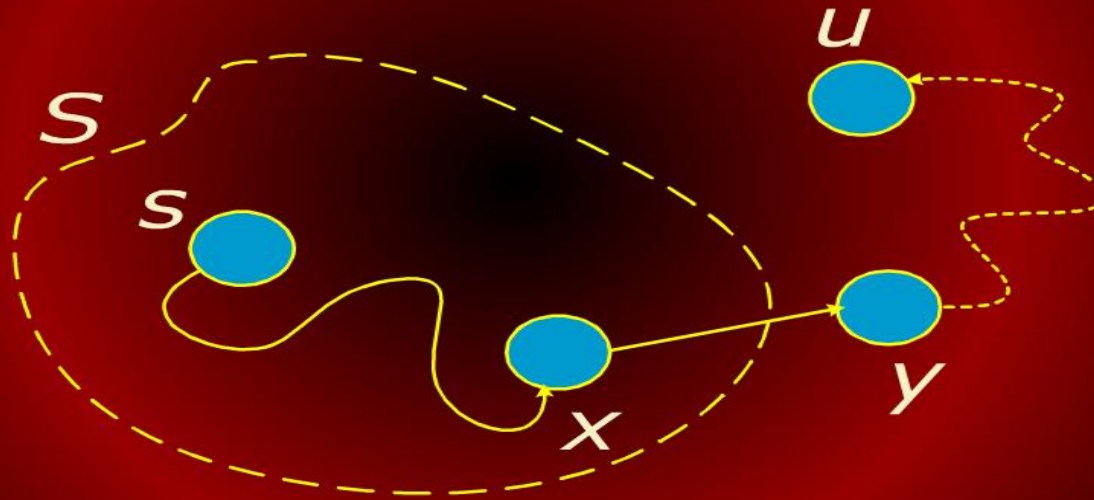
Dijkstra's Algorithm - Correctness

Suppose that $d(u) > \delta(s, u)$.

- The shortest path from s to u , $p(s, u)$, must pass through one or more vertices exterior to S .
- Let x be the last vertex inside S and y be the first vertex outside S on this path to u .
- Then
$$p(s, u) = p(s, x) \cup \{(x, y)\} \cup p(y, u).$$

Dijkstra's Algorithm – Correctness

Dijkstra's Algorithm - Correctness



Kruskal's Algorithm

Kruskal's Algorithm

- The length of $p(s, u)$ is $\delta(s, u) = \delta(s, y) + \delta(y, u)$.
- Because y was relaxed when x was put into S , $d(y) = \delta(s, y)$ by the convergence property.
- Thus $d(y) \leq \delta(s, u) \leq d(u)$.
- But, because $d(u)$ is the smallest among vertices not in S , $d(u) \leq d(y)$.
- The only possibility is $d(u) = d(y)$, which requires $d(u) = \delta(s, u)$ contradicting the assumption.